Exploratory Analysis and Interactive Visualization of COVID-19 Public Health Data

Dr. N. Sunanda
Associate Professor,
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Bowrampet, Hyderabad–500043, Telangana, India

K. Vishnu Vardhan
Department of CSE – Data Science
Koneru Lakshmaiah University
Hyderabad, Telangana, India
University ID: 2420030515

V. Sai Dhanush
Department of CSE – Data Science
Koneru Lakshmaiah University
Hyderabad, Telangana, India
University ID: 2420030162

***Abstract*—The COVID-19 pandemic generated large volumes of public health information distributed across national, state, district, daily, and testing datasets. Interpreting these records through raw tables alone can make it difficult to compare geographical areas, identify temporal changes, and understand the quality and coverage of the available data. This paper presents a data-science workflow for exploratory analysis and interactive visualization of COVID-19 public health data. The workflow combines CSV data ingestion, structural inspection, data cleaning, transformation, exploratory data analysis, statistical aggregation, visualization, and dashboard-oriented presentation. The supplied project implementation materials identify eight CSV datasets and report 262,597 records across the files. The displayed data-quality summary reports three files with no missing cells, five files containing missing values, and 37 duplicate rows detected. A state-level comparison chart presents confirmed, active, recovered, and deceased case categories for Maharashtra, Tamil Nadu, Andhra Pradesh, Karnataka, and Uttar Pradesh. A proposed architecture connects public COVID-19 CSV sources to preprocessing and analytical modules, a Python backend API, and a React dashboard. The paper describes the workflow, interprets the supplied visual evidence, discusses data-quality and reporting limitations, and identifies extensions for more reliable and accessible public-health data exploration. The dashboard is intended as an analytical and communication aid; it does not provide clinical advice or establish causal explanations for observed trends. In addition to describing the charts, the study explains how records with different geographical and temporal granularities can be inspected without assuming that they are directly interchangeable. It emphasizes the importance of retaining source-level metadata, distinguishing cumulative indicators from daily observations, and communicating missingness rather than concealing it through automatic replacement. The analysis is descriptive: the supplied screenshots and project summary are treated as evidence for the reported dataset counts and displayed comparisons, while implementation details that are not visible are identified as items requiring verification. The paper further outlines a practical user workflow, validation checks, reproducibility measures, privacy considerations, and design recommendations for making a public-health dashboard understandable to students, analysts, and general users. These additions provide a more complete account of the project lifecycle, from reading source files to presenting filtered results, while maintaining a clear boundary between demonstrated outputs and proposed system capabilities.**

***Index Terms*—COVID-19, public health data, exploratory data analysis, data preprocessing, data visualization, Python, React dashboard, CSV datasets, statistical analysis**

Course: 24AD3107 Data Science, Batch No. 2, Section 8.

I. Introduction

The COVID-19 pandemic created an exceptional demand for timely, interpretable public-health information. Governments, research groups, and public data platforms published information about confirmed cases, deaths, recoveries, testing, vaccination, and geographical distribution. These observations are valuable for understanding the evolution of the pandemic, but their usefulness depends on the ability to organize and interpret them correctly. Large datasets containing daily and regional records can be difficult to inspect manually, particularly when several files use different levels of aggregation or reporting formats.

Data science provides a structured approach for converting raw records into interpretable information. Data ingestion makes source files available for analysis; preprocessing addresses data-quality issues; exploratory data analysis (EDA) examines distributions and relationships; and visualization communicates selected patterns. Interactive dashboards add filtering and navigation so that users can inspect subsets of the data without repeatedly manipulating source files.

The project described in this paper focuses on COVID-19 data in India, with national, state, district, and testing-related CSV files. The supplied architecture identifies a Python analytical/backend layer and a React-based dashboard layer. The dashboard is designed to present summary statistics, daily trends, state comparisons, and region/date-based filtering. The paper documents the supplied implementation evidence and distinguishes reported screenshot metrics from analytical interpretations.

The principal contributions are:

- an end-to-end workflow for ingestion, cleaning, transformation, EDA, and visualization of COVID-19 CSV datasets;
- a modular architecture linking data processing, statistical analysis, backend API endpoints, and a React dashboard;
- a discussion of the supplied data-quality summary and state-wise case comparison;
- a structured account of limitations and practical extensions for dependable public-health dashboards.

II. Related Work

Dong, Du, and Gardner introduced an interactive web-based dashboard for tracking COVID-19 in real time. Their work demonstrated the value of bringing geographically organized case information into an accessible visual interface during a rapidly evolving public-health emergency [1]. The present project follows the same broad principle of making public-health data easier to explore, while concentrating on an India-oriented collection of CSV files and a modular dashboard architecture.

The World Health Organization (WHO) provides a centralized COVID-19 dashboard for reported cases, deaths, and vaccine utilization. WHO emphasizes that reported counts can vary because of differences in case definitions, testing strategies, reporting practices, and data cut-off times [2], [3]. These cautions are important when comparing regions or interpreting apparent changes in a time series. A visualization should therefore communicate the source, reporting period, and limitations of its data.

Our World in Data aggregates and documents COVID-19 indicators from multiple sources and provides downloadable data and visualizations [4]. Its data documentation also highlights the importance of checking the underlying source and license for third-party data [5]. Research on COVID-19 testing databases further demonstrates the value of documenting collection methods and source coverage when comparing testing activity across locations [6]. These works support the need for transparent dataset descriptions and clear definitions of the variables used in analysis.

The current project applies these principles at an exploratory level. It combines multiple CSV inputs, applies data-quality checks, groups observations by available geographical and temporal attributes, and presents the results through charts and dashboard views. Work on interactive exploratory visualization emphasizes that interaction can support analysis as well as communication [7], [15], [16]. Established visualization guidance also stresses choosing visual encodings that make comparisons perceptually clear [17], [18]. Unlike epidemiological modelling studies, the project does not claim to predict disease spread or infer causal relationships. Its primary goal is descriptive analysis and accessible data presentation.

III. Dataset and Data Collection

A. Dataset Overview

The supplied methodology diagram identifies eight CSV inputs: `complete.csv`, `state_level_daily.csv`, `state_level_latest.csv`, `district_level_latest.csv`, `nation_level_daily.csv`, `tests_day_wise.csv`, `tests_state_wise.csv`, and `patients_data.csv`. Together, these files represent different analytical views of COVID-19 information, including daily national and state observations, latest state and district summaries, testing information, and patient-related records.

The supplied results slide reports eight CSV datasets loaded and 262,597 total records across files. The exact number of

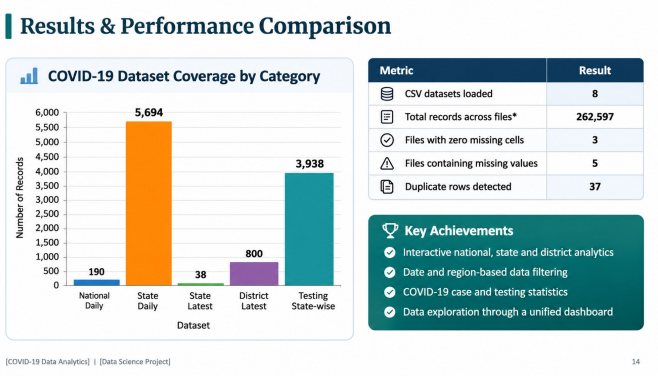


Fig. 1. Dataset coverage and project data-quality summary reproduced from the supplied project results slide.

columns, date coverage, source publication details, and access date are not visible in the supplied material. These metadata should be recorded from the actual files and their source documentation in a reproducible release. The eight inputs should not automatically be treated as one homogeneous table: each file can have a different row granularity, schema, and reporting purpose.

B. Data Ingestion

The ingestion stage reads the CSV files into a data-processing environment. A practical implementation can use the Python Pandas library to load each file into a separate DataFrame [20]. Matplotlib and Seaborn provide complementary facilities for static statistical graphics and distribution-oriented plots [21], [22]. Keeping the source datasets separate during initial inspection preserves their original structure and helps prevent accidental mixing of national, state, district, testing, and patient-level records.

For every input file, the ingestion stage should record its filename, number of rows, number of columns, column names, inferred data types, and missing-value count. Date columns should be parsed into a consistent date representation. Numeric columns should be checked for valid numeric values, and categorical columns should be examined for inconsistent spelling, spacing, or capitalization. The resulting metadata can be used to validate later transformations and dashboard filters.

C. Dataset Coverage

Fig. 1 reproduces the dataset coverage chart supplied with the project presentation. It displays record counts for five dataset categories: National Daily (190), State Daily (5,694), State Latest (38), District Latest (800), and Testing State-wise (3,938). These category counts describe the values displayed in the supplied chart and should be interpreted as coverage counts for those named categories, not as a replacement for the overall total of 262,597 records reported across all eight CSV files. The difference in scale indicates that the files serve different levels of detail and analytical purposes.

IV. Proposed Methodology and System Architecture

A. Overall Workflow

The proposed methodology follows a sequence of data collection, preprocessing, analytical processing, statistical examination, backend service generation, and dashboard presentation. Fig. 2 presents the architecture supplied with the project. It separates the system into six logical layers: data sources; data ingestion and preprocessing; analytical processing; statistical techniques; API/backend; and React dashboard.

The separation of these responsibilities supports maintainability. Data ingestion and cleaning can be updated without redesigning the dashboard, while additional charts can be introduced without changing the source-file format. The architecture also makes the data flow understandable to developers and project evaluators.

B. Data Cleaning and Transformation

The preprocessing stage checks missing values, duplicate records, inconsistent values, and incorrect data formats. Missing-value treatment must depend on the meaning of the attribute. A missing case count should not automatically be replaced with zero, because zero represents an observed count while a missing value indicates unavailable information. The selected treatment should be recorded for each relevant column.

Duplicate detection should consider the fields that uniquely identify an observation. Two rows with the same case count may still represent different dates or locations and should not be removed solely because some values match. Where duplicate rows are confirmed, the cleaning process should retain a record of the number removed.

Data transformation includes parsing dates, converting numeric fields, standardizing state or district labels, and preparing consistent fields for aggregation. Before combining data, the analyst should verify that the files use compatible definitions and time intervals. For example, a daily count and a cumulative count are different measures and must not be aggregated as though they were equivalent.

C. Exploratory Data Analysis

EDA is performed on the cleaned data to understand its structure and identify useful patterns. Descriptive statistics such as count, mean, median, minimum, maximum, and standard deviation can summarize numerical attributes. Frequency counts can describe categorical attributes such as state or district. Grouping and aggregation can produce totals and averages for selected dates or regions. This exploratory approach follows the broader philosophy of exploratory data analysis described by Tukey [19].

Temporal analysis examines changes over time where valid date fields are available. Regional analysis compares state or district values over a consistent period. Testing analysis examines the available testing records separately from case counts. Correlation analysis may be considered only when the selected variables are suitable and the relationship can be interpreted responsibly. Correlation alone does not establish causation.

D. Statistical and Comparative Techniques

The architecture identifies descriptive statistics, grouping and aggregation, percentage difference, correlation analysis where suitable, and time-series visualization. For two comparable regional values, percentage difference can be calculated using a clearly defined denominator. One possible relative-change expression is

$$P = \frac{V_2 - V_1}{V_1} \times 100, \tag{1}$$

where V_1 is the reference value and V_2 is the comparison value. If $V_1 = 0$, the percentage expression is undefined and should not be displayed as an ordinary percentage.

E. Backend and Dashboard Integration

The supplied architecture proposes a Python backend API and a React frontend. The API layer is intended to return filtered and aggregated data in JSON form. The diagram lists example endpoints `/api/states`, `/api/summary`, `/api/trend`, and `/api/compare`. These endpoint names are represented as part of the supplied architecture; the final implementation report should verify the exact routes and response schemas against the running application.

The React dashboard is designed around summary cards, state and date filters, trend charts, regional comparison charts, and insight/report views. The same filter selection should be applied consistently to all dependent charts and summary metrics. The interface should also show the selected date range and geographical level so that users understand which subset of the data is currently displayed.

V. Data Visualization and Dashboard Design

A. State-wise Case Comparison

Fig. 3 shows the supplied grouped bar chart for Maharashtra, Tamil Nadu, Andhra Pradesh, Karnataka, and Uttar Pradesh. Four categories are represented: confirmed, active, recovered, and deceased. The chart enables a visual comparison of the displayed state-level values and shows that the magnitude of confirmed cases differs across the selected states. Maharashtra has the tallest confirmed and recovered bars in this plotted selection. The chart also shows active and deceased categories separately, avoiding the assumption that total confirmed cases alone describe the current status of a region.

The chart is based on the supplied screenshot, which shows the code selecting rows from a DataFrame and plotting a grouped bar chart. The screenshot does not show the underlying CSV file, the exact date represented, or the full numerical table. Therefore, the interpretation here is limited to the displayed comparison. A final reproducible paper should include the source date, units, and exact values used to generate the figure.

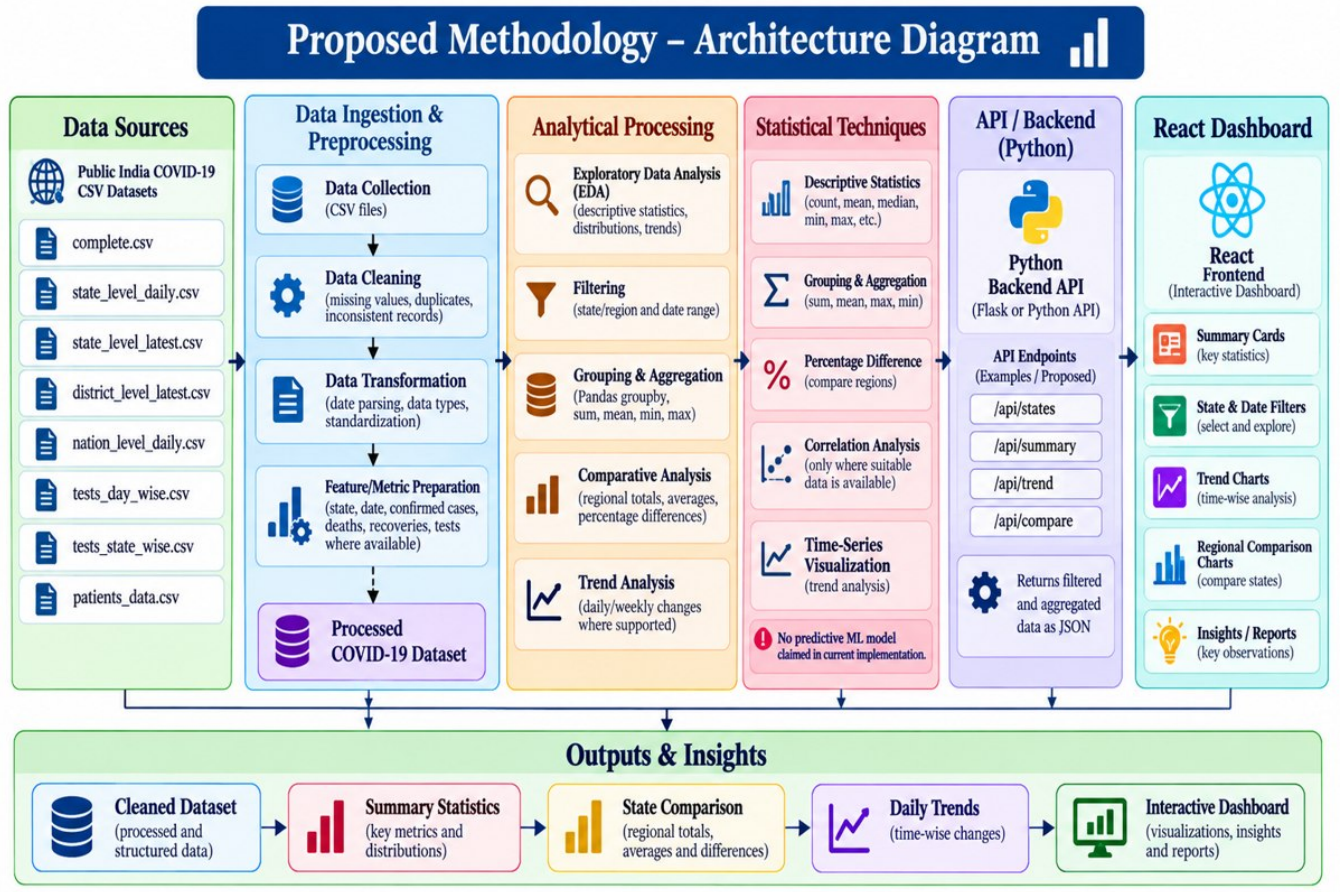


Fig. 2. Proposed methodology and system architecture showing CSV sources, preprocessing, analytical modules, statistical techniques, Python backend API, and React dashboard.

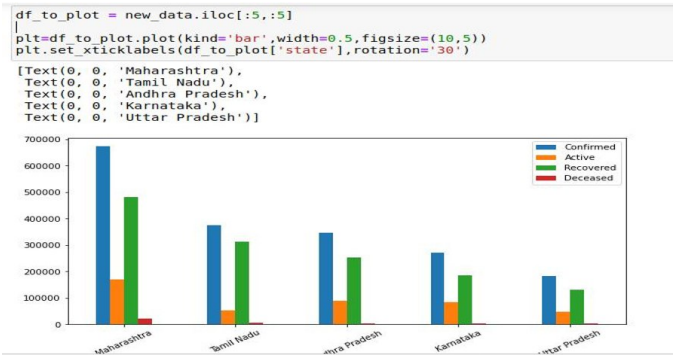


Fig. 3. Grouped bar chart comparing confirmed, active, recovered, and deceased COVID-19 values across five Indian states.

B. Trend and Regional Views

A line chart is appropriate for displaying a time-ordered measure such as daily confirmed cases or daily testing counts. The x-axis should represent the date and the y-axis should identify the selected measure and its unit. Where the source data contain cumulative totals, the chart title and axis label should explicitly state that the values are cumulative. A bar chart is more appropriate for comparing discrete regions or categories at a selected point in time.

The dashboard architecture also identifies regional comparison charts and insight/report views. A useful comparison should use the same reporting period and compatible measures for every region. If one state has missing observations or a different reporting interval, the interface should make that limitation visible rather than silently treating the records as equivalent.

C. Dashboard Data Flow

Fig. 4 presents the supplied data-flow diagram. COVID-19 source data enter the dashboard process, where records are parsed and aggregated. The dashboard exchanges data with a dataset store, receives user requests and filter selections, and returns charts, statistics, and visual insights. This representation highlights the interactive nature of the system: the user selects a region or timeframe, the application retrieves the corresponding data, and the interface updates the displayed information.

D. Visualization Quality

Every chart should contain a descriptive title, readable axis labels, consistent units, and a legend when multiple series are

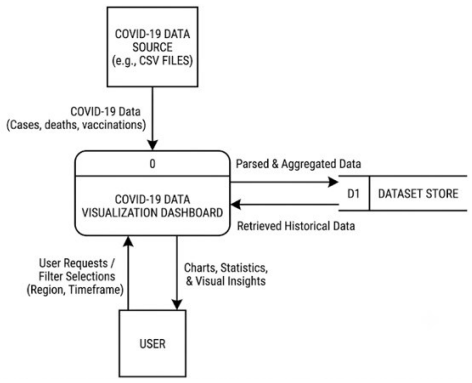


Fig. 4. Dashboard data-flow diagram showing source data, processing, dataset storage, user filter requests, and visual outputs.

TABLE I
DATA INGESTION AND QUALITY METRICS SHOWN IN THE SUPPLIED RESULTS SLIDE

Metric	Reported result
CSV datasets loaded	8
Total records across files	262,597
Files with zero missing cells	3
Files containing missing values	5
Duplicate rows detected	37

displayed. State names should be rotated or wrapped when necessary to prevent overlap. The scale should be selected to preserve meaningful differences without exaggerating small variations. When the dashboard applies filters, the selected filters should remain visible and should be applied consistently across all related visualizations.

VI. Results and Discussion

A. Dataset and Data-quality Results

The supplied performance summary reports the following metrics: eight CSV datasets loaded, 262,597 total records across files, three files with zero missing cells, five files containing missing values, and 37 duplicate rows detected. These values are transcribed from the project results slide. They provide a concise overview of the ingestion and initial quality-checking stage.

The metrics indicate that missing-value handling is relevant to the workflow because five of the eight files are reported to contain missing values. The 37 duplicate rows indicate that duplicate detection was performed, although the screenshot does not specify the files or columns in which duplicates were found, nor whether all detected rows were removed. The final implementation should preserve before-and-after row counts and document the treatment applied to each affected dataset.

B. State Comparison Findings

The state comparison chart provides a compact view of four COVID-19 indicators across five states. The confirmed and recovered bars are visibly larger for Maharashtra than for the other states in the displayed selection. Tamil Nadu and Andhra Pradesh also show substantial confirmed and recovered values, while Karnataka and Uttar Pradesh appear lower in the chart. Active and deceased values are shown as separate categories and should be interpreted according to the same date and data definitions as the confirmed and recovered measures.

These visual observations should not be generalized to all dates or to the full history of the pandemic. A ranking can change when the reporting date, population size, or aggregation method changes. For stronger comparisons, the dashboard can provide both absolute counts and population-normalized rates, provided that reliable population denominators are available and their sources are cited.

C. System-level Results

The supplied architecture and dashboard result slide identify interactive national, state, and district analytics; date- and region-based filtering; COVID-19 case and testing statistics; and data exploration through a unified dashboard as key achievements. The slide also identifies the planned use of summary cards, trend charts, regional comparison charts, and insight/report views. These are reported as project achievements in the supplied material. Screenshots of the actual running interface and API responses should be retained as implementation evidence for final evaluation.

D. Discussion

The combined results show the value of organizing multiple public-health files into a common analytical workflow. The dataset coverage chart makes the differing sizes of national, state, district, and testing datasets visible. The quality summary highlights the need to inspect missing values and duplicates before calculating regional totals. The state comparison chart demonstrates how a grouped visualization can make several indicators easier to compare than a raw table.

At the same time, the results have limitations. The screenshots do not expose the complete schemas, source URLs, date coverage, exact state-wise values, or all preprocessing decisions. Counts from different CSV files may overlap or describe different reporting levels, so their row totals should not be added together as if they represented unique people or unique events. Likewise, confirmed, active, recovered, and deceased values may be defined differently across sources or dates. The dashboard should communicate these definitions and avoid unsupported epidemiological conclusions.

VII. Data Validation and Reproducibility

A. Validation at File Level

A dependable analytical pipeline should validate each source file before performing any aggregation. File-level validation begins by confirming that the expected file is present, can be opened, and contains a non-empty table. The pipeline should then inspect the header row and compare the observed column names with an expected schema. This check helps detect renamed columns, unexpected delimiters, encoding problems, and changes introduced by a revised data release. A validation report can store the filename, file size, row

count, column count, detected data types, and processing timestamp. These fields make it easier to identify whether a later result differs because the source itself changed or because the analysis code changed.

Date validation is particularly important for public-health datasets. A date field should be parsed using an explicit format where possible, and invalid values should be counted rather than silently discarded. The minimum and maximum valid dates can be reported for each dataset. If a daily dataset is expected, repeated dates may be legitimate when each row represents a different state or district; consequently, uniqueness must be checked using a composite key such as location plus date, not date alone. The precise key depends on the schema and should be documented after inspecting the original files.

Numeric validation should check whether case and testing fields can be converted to numeric values and whether impossible negative values or unexpected text tokens occur. Such checks do not automatically prove that a value is incorrect; they flag observations for review against the source definition. Categorical validation should inspect spelling variants, leading or trailing spaces, abbreviations, and changes in administrative names. A mapping table can standardize labels while preserving the original source value in a separate field. This approach supports consistent grouping without losing traceability.

B. Validation Across Data Processing Stages

Validation should continue after cleaning and transformation. A useful practice is to record row counts at three points: before cleaning, after cleaning, and after aggregation. If rows are removed, the report should state why they were removed. If missing values are retained, the report should indicate which calculations exclude them. If a dataset is joined to another dataset, the join type and expected relationship should be stated. For example, a many-to-one join between district observations and a state lookup table should not unexpectedly multiply the number of observations.

The project summary reports 37 duplicate rows detected, but the available screenshot does not identify the affected files or the rule used to define a duplicate. A reproducible implementation should therefore report both the number of exact duplicate rows and the number of key collisions, where applicable. Exact duplicates are rows that match across all selected columns; key collisions share the fields intended to identify an observation. The two conditions are not necessarily identical. This distinction prevents valid records from being deleted simply because they share a subset of values.

A compact validation log can include the following fields: source filename, original row count, missing-cell count, exact duplicate count, invalid-date count, invalid-numeric count, cleaned row count, and final aggregation row count. The log should be generated automatically and retained with the output charts. If the data are refreshed, the same checks can be repeated and compared with the previous run. This process improves auditability and makes the dashboard easier to maintain.

C. Reproducible Analytical Workflow

Reproducibility requires more than retaining the final chart image. The analyst should preserve the source files or their stable download references, the processing script, library versions, configuration values, and the date on which the analysis was performed. A fixed sequence of operations should be used: load the data, inspect schemas, normalize types, apply documented cleaning rules, compute aggregates, and generate visualizations. Each stage should consume a clearly defined input and produce a clearly defined output.

The same filter parameters should produce the same aggregate values when the source files and code version remain unchanged. This can be checked by saving a small set of test queries, such as a national summary for a known date or a state comparison for a fixed group of states. These checks are not substitutes for full software testing, but they provide useful regression tests when the pipeline is modified. The project can also record a checksum for each source file so that a changed file can be identified even when its filename remains the same.

VIII. Dashboard Interaction and User Workflow

A. Typical User Journey

The dashboard is intended to reduce the effort required to move between raw CSV files and interpretable summaries. A typical user begins with a national overview containing a small number of summary indicators. The user can then select a geographical level, such as state or district, and choose a reporting period. The interface should display the active selections near the charts so that the context of every displayed value remains clear. When a user changes a filter, all dependent cards and charts should update together; otherwise, the dashboard may present contradictory summaries.

A state-comparison task can begin by selecting a reporting date and choosing several states. The application can then display confirmed, active, recovered, and deceased values in a grouped chart. A trend task can use the same state filter but display a time series across the selected period. A testing task should use the testing datasets and clearly label the selected measure. These workflows show why the dashboard should keep data definitions and geographical level visible rather than relying only on chart titles.

B. Dashboard Components

Summary cards are useful for presenting a small number of high-level indicators, but each card should identify the metric, unit, date, and geographical scope. A card showing a cumulative count should not be visually or verbally confused with a daily count. Trend charts should include readable date ticks and should avoid overcrowding the horizontal axis. Regional comparison charts should use a consistent ordering and scale where the purpose is direct comparison. Tooltips can expose exact values without placing every label directly on the chart.

Filters should have predictable behavior. A date-range selector should prevent an end date earlier than the start date. A state selector should not offer district values from unrelated

states unless the user explicitly chooses a national district view. A reset action can restore the default selection. If a filter produces no records, the interface should show a clear empty-state message rather than a blank chart that may be mistaken for a zero value. If some records are missing, the interface should distinguish unavailable data from a measured zero.

C. Accessibility and Communication

A public-health dashboard may be used by people with different levels of statistical and technical knowledge. Plain-language labels can reduce ambiguity, while short explanatory notes can define terms such as active cases, cumulative cases, and testing counts according to the selected source. Color should not be the only way to distinguish chart series; labels, marker shapes, or line styles can provide additional cues. Text and chart contrast should remain readable, and interactive controls should have descriptive names.

The dashboard should also support users who need to inspect exact values. Tables or downloadable filtered records can complement charts, particularly when a user needs to cite a value in a report. Each visualization should include a source note and the date on which the data were last refreshed. When data are revised by the original publisher, the interface should communicate the new retrieval date and, where practical, whether historical values have changed. These measures help prevent a screenshot from being interpreted as a timeless or universally complete account of the pandemic.

IX. Privacy, Responsible Use, and Operational Considerations

A. Privacy and Data Minimization

The listed files include a patient-related dataset. Patient-level records require greater care than aggregated public statistics because combinations of attributes may reveal information about individuals even when direct identifiers are absent. The dashboard should therefore prioritize aggregated views and should not expose personal-level records unless there is a clear, authorized purpose and an appropriate privacy review. Direct identifiers should not be included in public charts, logs, downloadable outputs, or demonstration screenshots. Access to any restricted data should be controlled separately from access to public aggregate summaries.

Data minimization is a practical design principle: only fields needed for the intended analysis should be loaded into the dashboard-facing layer. If patient-level details are not required for the published analysis, they should remain outside the public-facing service. The project should document the source license and applicable terms of use before redistributing data. This paper does not claim that a formal privacy assessment or legal review has been completed; these are necessary governance steps for any deployment that handles potentially sensitive records.

B. Responsible Interpretation

COVID-19 indicators are affected by testing availability, reporting delays, changes in definitions, and retrospective corrections. A rise in reported cases may reflect a change in testing or reporting as well as a change in transmission. Similarly, comparisons of absolute counts across states are influenced by population size. The dashboard should therefore avoid labels such as safest or worst based only on raw totals. Where appropriate denominators are available, rates may provide additional context, but the denominator source and reference date must be documented.

The visualizations in this paper are descriptive and should be interpreted within the limitations of the supplied data. They do not establish that a policy caused a change in cases, nor do they predict future infections. Any future forecasting module would require a separate methodology, out-of-sample evaluation, uncertainty intervals, and careful explanation of assumptions. Keeping descriptive summaries separate from predictive outputs reduces the risk that users mistake an exploratory chart for a clinical or policy recommendation.

C. Operational Performance and Maintenance

As the number of source files and dashboard users grows, the system should avoid repeatedly performing expensive cleaning operations for every interaction. A practical design is to run ingestion and preprocessing as a scheduled or manually triggered batch process, store validated analytical tables, and allow the API to retrieve precomputed summaries. The dashboard can then request only the subset needed for the selected region and period. This separation can improve response time and make data refreshes easier to monitor.

Operational monitoring should record whether each source file was retrieved successfully, whether schema validation passed, how many rows were processed, and whether the dashboard API returned an expected response. Error messages should be informative without exposing file-system paths, credentials, or sensitive records. The project architecture identifies a Python backend and React frontend, but the supplied evidence does not include measured response times, concurrent-user tests, or deployment configuration. Such performance claims should be added only after controlled testing.

X. Limitations and Threats to Validity

The current evidence is primarily based on the supplied project screenshots and paper content. The exact source publisher and access date for each CSV file are not specified in the screenshots. The paper therefore reports only the filenames and summary metrics visible in the project material. The exact data dictionary, date range, record-level deduplication key, missing-value strategy, and validation rules must be documented from the implementation.

COVID-19 reporting practices have changed over time. WHO notes that differences in case definitions, testing strategies, reporting practices, and reporting frequency can affect comparisons across sources and periods [2], [3]. A dashboard that combines records from different sources must therefore preserve source metadata and make the selected reporting period clear. A visual association between two variables should not be interpreted as a causal relationship.

XI. Conclusion

This paper presented an exploratory data analysis and interactive visualization workflow for COVID-19 public-health information. The project combines eight CSV inputs with a reported total of 262,597 records, performs data-quality checks, supports statistical and regional analysis, and organizes the outputs through a Python backend and React dashboard architecture. The supplied results report three files with no missing cells, five files with missing values, and 37 duplicate rows detected. A state-wise grouped bar chart illustrates comparisons among confirmed, active, recovered, and deceased categories for five Indian states.

The architecture provides a clear separation between data ingestion, preprocessing, analysis, API services, and dashboard presentation. The value of the system lies in making public-health data easier to inspect and compare. Its reliability depends on transparent source documentation, appropriate cleaning decisions, consistent aggregation, and careful interpretation of reporting differences. Future versions should attach exact dataset metadata and reproducible processing records to every chart and dashboard view.

XII. Future Scope

The project can be extended in several directions:

- **Automated data refresh:** schedule ingestion from maintained public datasets and record the retrieval timestamp and source version.
- **Improved data validation:** add schema checks, date-range validation, state/district name standardization, and duplicate-key rules.
- **Population-normalized comparison:** provide rates per population where a trustworthy denominator is available.
- **Enhanced dashboard interaction:** add downloadable filtered data, clear filter summaries, and accessible chart descriptions.
- **Uncertainty and provenance:** display source, reporting frequency, update time, and known limitations alongside every chart.
- **Analytical extensions:** introduce forecasting or machine-learning experiments only as separately validated modules, with suitable evaluation and clear separation from descriptive statistics.

These extensions should be implemented only when suitable data and validation procedures are available. Predictive functionality is not claimed as part of the current descriptive workflow.

Acknowledgment

The authors acknowledge the guidance and support provided by Dr. N. Sunanda during the development of the project. The authors also acknowledge the academic support provided by the Department of CSE – Data Science, Koneru Lakshmaiah University.

References

[1] E. Dong, H. Du, and L. Gardner, "An interactive web-based dashboard to track COVID-19 in real time," *The Lancet Infectious Diseases*, vol. 20, no. 5, pp. 533–534, May 2020, doi: 10.1016/S1473-3099(20)30120-1.

[2] World Health Organization, "About the WHO COVID-19 dashboard," WHO Health Emergencies Programme. [Online]. Available: https://data.who.int/dashboards/covid19/about. Accessed: Nov. 5, 2026.

[3] World Health Organization, "WHO COVID-19 dashboard data," WHO Health Emergencies Programme. [Online]. Available: https://data.who.int/dashboards/covid19/data. Accessed: Nov. 5, 2026.

[4] E. Mathieu, H. Ritchie, L. Rodés-Guirao, C. Appel, D. Gavrilov, C. Giattino, J. Hasell, B. Macdonald, S. Dattani, E. Ortiz-Ospina, and M. Roser, "COVID-19 pandemic," *Our World in Data*, 2020. [Online]. Available: https://ourworldindata.org/coronavirus. Accessed: Nov. 5, 2026.

[5] Our World in Data, "COVID-19 data repository: Public data documentation," GitHub repository. [Online]. Available: https://github.com/owid/covid-19-data/tree/master/public/data. Accessed: Nov. 5, 2026.

[6] J. Hasell, E. Mathieu, D. Beltekian, B. Macdonald, C. Giattino, Y. Ortiz-Ospina, M. Roser, and H. Ritchie, "A cross-country database of COVID-19 testing," *Scientific Data*, vol. 7, Art. no. 345, 2020, doi: 10.1038/s41597-020-00688-8.

[7] H. S. Badr, H. Du, M. Marshall, E. Dong, M. M. Squire, and L. M. Gardner, "Association between mobility patterns and COVID-19 transmission in the USA: A mathematical modelling study," *The Lancet Infectious Diseases*, vol. 20, no. 11, pp. 1247–1254, Nov. 2020, doi: 10.1016/S1473-3099(20)30553-3.

[8] V. Vasudevan, A. Gnanasekaran, V. Sankar, S. A. Vasudevan, and J. Zou, "Disparity in the quality of COVID-19 data reporting across India," *BMC Public Health*, vol. 21, Art. no. 1211, Jun. 2021, doi: 10.1186/s12889-021-11054-7.

[9] V. Vasudevan, A. Gnanasekaran, V. Sankar, S. A. Vasudevan, and J. Zou, "Variation in COVID-19 data reporting across India: 6 months into the pandemic," *Journal of the Indian Institute of Science*, vol. 100, no. 4, pp. 885–892, 2020, doi: 10.1007/s41745-020-00188-z.

[10] V. Vasudevan, A. Gnanasekaran, B. Bansal, C. Lahariya, G. G. Parameswaran, and J. Zou, "Assessment of COVID-19 data reporting in 100+ websites and apps in India," *PLOS Global Public Health*, vol. 2, no. 4, Art. no. e0000329, 2022, doi: 10.1371/journal.pgph.0000329.

[11] Y. Rimal, S. Gochhait, and A. Bisht, "Data interpretation and visualization of COVID-19 cases using R programming," *Informatics in Medicine Unlocked*, vol. 26, Art. no. 100705, 2021, doi: 10.1016/j.imu.2021.100705.

[12] M.-F. Pang, Z.-R. Liang, Z.-D. Cheng, X.-P. Yang, J.-W. Wu, K. Lyu, J.-J. Xi, Z.-J. Li, G.-Q. Shi, Y.-P. Zhang, G. F. Gao, X.-P. Qi, and X.-P. Dong, "Spatiotemporal visualization for the global COVID-19 surveillance by balloon chart," *Infectious Diseases of Poverty*, vol. 10, Art. no. 21, 2021, doi: 10.1186/s40249-021-00800-z.

[13] C. G. Healey, S. J. Simmons, C. Manivannan, and Y. Ro, "Visual analytics for the coronavirus COVID-19 pandemic," *Big Data*, vol. 10, no. 2, 2022, doi: 10.1089/big.2021.0023.

[14] P. Bankar, S. Panda, V. Anand, and V. Kumar, "Risk and survival analysis from COVID outbreak data: Lessons from India," in *Proc. AAAI Spring Symp. Survival Prediction*, Proc. Mach. Learn. Res., vol. 146, pp. 215–222, 2021.

[15] J. A. Dixon *et al.*, "Leveraging data visualization and a statewide health information exchange to support COVID-19 surveillance and response: Application of public health informatics," *Journal of the American Medical Informatics Association*, vol. 28, no. 7, pp. 1363–1373, 2021.

[16] S. Ngai *et al.*, "Built by epidemiologists for epidemiologists: An internal COVID-19 dashboard for real-time situational awareness in New York City," *JAMIA Open*, vol. 5, no. 2, Art. no. ooac029, 2022, doi: 10.1093/jamiaopen/ooac029.

[17] E. G. Wettstone *et al.*, "Interactive SARS-CoV-2 dashboard for real-time geospatial visualisation of sewage and clinical surveillance data from Dhaka, Bangladesh: A tool for public health situational awareness," *BMJ Global Health*, 2023, doi: 10.1136/bmjgh-2023-012921.

[18] M. Gupta *et al.*, "Tracing the COVID-19 spread pattern in India through a GIS-based spatio-temporal analysis of interconnected clusters," *Scientific Reports*, vol. 14, Art. no. 847, 2024, doi: 10.1038/s41598-023-50933-4.

[19] K. Imdad, M. Sahana, M. J. Rana, I. Haque, P. P. Patel, and M. Pramanik, "A district-level susceptibility and vulnerability assessment of the COVID-19 pandemic's footprint in India," *Spatial and*

Spatio-temporal Epidemiology, vol. 36, Art. no. 100390, 2021, doi: 10.1016/j.sste.2020.100390.
[20] S. Guchhait, S. Das, N. Das, and T. Patra, "Mapping of space-time patterns of infectious disease using spatial statistical models: A case study of COVID-19 in India," *Infectious Diseases*, vol. 55, no. 1, pp. 27–43, 2023, doi: 10.1080/23744235.2022.2129778.
[21] W. Wang, J. Blossom, J. Kim, P. deSouza, W. Zhang, R. Kim, R. Sarwal, and S. V. Subramanian, "COVID-19 metrics across parliamentary constituencies and districts in India," *Annals of GIS*, vol. 28, no. 4, pp. 435–443, 2022, doi: 10.1080/19475683.2022.2044903.
[22] B. Pandey, J. Gu, and A. Ramaswami, "Characterizing COVID-19 waves in urban and rural districts of India," *npj Urban Sustainability*, vol. 2, Art. no. 26, 2022.
[23] "Geographic information system-based analysis of COVID-19 cases in India during pre-lockdown, lockdown, and unlock phases," *International Journal of Infectious Diseases*, vol. 105, pp. 424–435, 2021, doi: 10.1016/j.ijid.2021.02.070.
[24] C. Costa-Santos *et al.*, "COVID-19 surveillance data quality issues: A national consecutive case series," *BMJ Open*, vol. 11, no. 12, Art. no. e047623, 2021, doi: 10.1136/bmjopen-2020-047623.
[25] A. Batch and N. Elmqvist, "The interactive visualization gap in initial exploratory data analysis," *IEEE Transactions on Visualization and Computer Graphics*, vol. 24, no. 1, pp. 278–287, 2018, doi: 10.1109/TVCG.2017.2743990.
[26] J. W. Tukey, *Exploratory Data Analysis*. Reading, MA, USA: Addison-Wesley, 1977.
[27] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf.*, 2010, pp. 56–61, doi: 10.25080/Majora-92bf1922-00a.
[28] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007, doi: 10.1109/MCSE.2007.55.
[29] M. L. Waskom, "seaborn: Statistical data visualization," *Journal of Open Source Software*, vol. 6, no. 60, Art. no. 3021, 2021, doi: 10.21105/joss.03021.
[30] W. S. Cleveland and R. McGill, "Graphical perception: Theory, experimentation, and application to the development of graphical methods," *Journal of the American Statistical Association*, vol. 79, no. 387, pp. 531–554, 1984, doi: 10.1080/01621459.1984.10478080.

Appendix: Indicator Definitions and Review Checklist

The following notes provide a concise interpretation guide for readers of the dashboard. They are not a replacement for the original data dictionary; the exact definition used by each source file must take precedence. *Confirmed cases* generally refers to the cumulative number of cases reported as confirmed by the source up to a specified date. *Active cases* is a source-defined status measure and should not be recalculated unless the source provides a consistent formula and all required components. *Recovered cases* and *deceased cases* are cumulative outcome indicators when supplied in that form. *Daily cases* represent newly reported observations for a date and must be distinguished from cumulative totals. Testing indicators may refer to samples, tests, or people tested; these units are not interchangeable and should be stated explicitly.

Before publishing a chart, the analyst should confirm that the selected source file and reporting date are known; the geographical level is consistent; the displayed measure and unit are stated; missing values are not represented as measured zeroes; duplicate handling is documented; and the aggregation method matches the source definition. For a comparison chart, all regions should use the same date or clearly stated time window. For a trend chart, the analyst should check date ordering, gaps, and whether values are daily or cumulative. For a percentage calculation, the denominator and treatment of zero values should be explained.

A final review should also confirm that the chart title describes the metric rather than implying a conclusion, the legend maps clearly to the displayed series, labels are readable at normal zoom, and colors remain distinguishable. The dashboard should identify the data source and last refresh date, show the active filters, and provide a clear message when no records match a selection. Downloaded tables should retain the date and region fields needed to interpret each row. These checks help ensure that the visualization remains transparent when reused in presentations, classroom demonstrations, or analytical reports.

Suggested Evidence to Retain

For an academic demonstration, the project team should retain a small evidence package alongside the paper. It can include a copy of each source file or a stable source reference, a data dictionary, the preprocessing script, a validation summary, and the final chart-generation script. The evidence package should identify the software environment and the date of execution. If a dashboard is demonstrated live, screenshots should show the selected filters and the resulting charts, while a separate record should capture the corresponding API request and response structure. Personal or restricted records should be excluded from these materials.

The evidence should also distinguish three kinds of statements. First, directly measured results are values calculated from the supplied data and should be accompanied by the relevant file, field, date range, and calculation. Second, visual observations describe what can be seen in a chart and should be limited to the plotted selection. Third, proposed capabilities describe what the architecture is designed to support but may not yet be verified in a running deployment. Keeping these categories separate improves the accuracy of a project report and helps reviewers reproduce the conclusions.

When the source data are updated, a new validation report should be generated rather than overwriting the earlier record. The team can compare row counts, date coverage, missingness, and duplicate counts between releases. If a change is large, it should be investigated before charts are refreshed. This simple release discipline makes it possible to explain why a figure in an earlier presentation may differ from a figure generated later. It also gives future students or developers a clear starting point for extending the project without relying on undocumented manual steps.